



Code evaluation and refactoring checklist for Pandas and Matplotlib

Part 0 — OOP & Code Quality (5 items)

- **Separation of concerns:** The student splits code into clear roles (generation, noise, aggregation, plotting, I/O).
- **Single responsibility:** Each class/module does one thing; there is no “god script.”
- **Reproducibility:** Randomness uses a **controllable seed** passed into functions/classes.
- **Data integrity:** The Customer_ID $\leftrightarrow$ Customer_Name mapping is enforced and consistent across files.
- **Error handling:** Predictable issues (no data, invalid dates) are handled with clear exceptions or friendly messages.

Part 1 — Dataset Crafting (daily_entries.csv) (5 items)

- **Customers & days:** The student creates **15 customers** and generates entries on **50 different valid days** across months.
- **Dates valid:** Day/month values are valid for each month (no impossible dates).
- **Values:** Each Value lies in **[-15.00, 15.00]** and is **rounded to 2 decimals**.
- **Schema:** The CSV has exactly: Customer_Name, Customer_ID, Day, Month, Value (with consistent types).
- **Determinism:** Running with the same seed reproduces the same dataset.

TODO:

- Write your short review about your peer’s code for this part.
 - Reimplement this part using different functions of Pandas for all the tasks.
-



Part 2 — Dataset Handling (A/B/C combined) (5 items)

- **Monthly balances:** The student groups by **Customer + Month**, sums values, names the sum clearly (e.g., `Monthly_Balance`), and saves `monthly_balances.csv`.
- **Noise injection:** A **second** dataset with the **same customers** is generated; NaN is injected thoughtfully (IDs are not broken) and saved as `daily_entries_2.csv`.
- **NaN policy:** For sums, **all-NaN groups do not become 0** (e.g., `sum(min_count=1)` or equivalent is used).
- **Update policy:** When updating `monthly_balances.csv` with the noisy file, a **documented policy** is implemented (**replace** or **add**) and applied consistently.
- **Combine smartly:** The two daily files are concatenated, **duplicates are avoided** (e.g., via a transaction ID or careful key), and saved as `daily_entries_total.csv`.

TODO:

- Write your short review about your peer's code for this part.
 - Reimplement this part using different functions of Pandas for all the tasks.
-

Part 3 — Data Plotting (from `daily_entries_total.csv`) (5 items)

- **By customer:** Given name or ID, all entries for that customer are plotted over time (clear labels, 0-line).
- **By day:** Given day+month, all entries on that day are plotted; **no-entries** cases are handled gracefully (message/exception).
- **Monthly balance:** For a customer, monthly totals are plotted in chronological order with sensible axis labels.
- **Range per customer:** Given start–end dates, entries within the period are shown using **grouped bars by customer**.
- **Negative-only:** Given start–end dates, **only negative transactions** in that period are plotted.

TODO:

- Write your short review about your peer's code for this part.
 - Reimplement this part using different functions of Pandas for all the tasks.
-



✓ **Tips (optional, for quick review)**

- Prefer a single internal date; derive Day/Month for CSV output.
- Keep file I/O **out of constructors**; perform it in explicit methods.
- Use a **unique Transaction_ID** to prevent double counting when combining.
- State assumptions clearly (seed used, update policy, NaN handling) in comments/README.
- Make plots readable (title, axes, legend, grid; months sorted chronologically).